claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-a027f9c07747f1ff3.jsonl`
- SHA-256: `bb8d935a9556af17ef2126f73aa501ed2d65f8b6fedf728415696e0edd260bb1`
- Source modified: `2026-07-02T03:04:40+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_41c8dc8a-8a4`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`

Transcript

1. user (2026-07-02T03:02:20.799Z)

```
You are an adversarial verifier in a tech-debt audit. Your default stance: REFUTE. Today is
2026-07-02.
A finder claims the following about the repo at C:/Users/avidu/Projects/khelsutra-guru/khelsutra
(avidullu/khelsutra):

{
  "title": "Issue #99 still valid: uploaded filename never shown during upload/processing",
  "category": "tech-debt",
  "severity": "P2",
  "file": "web/src/components/IngestPanel.tsx",
  "line": 306,
  "evidence": "useUpload carries the filename in its phase (useUpload.ts:47,58) but IngestPanel
shows only a static 'Uploading?' chip (IngestPanel.tsx:306; locales/en/common.json:95) and
useUpload.ts:62 resets to idle on completion ? the name is dropped exactly when engine
processing starts. The persistent current-video label (App.tsx:295-307) appears only after the
job is done and videoId is set.",
  "proposed_fix": "Quick win: surface upload.phase.filename in the uploading chip, and hold the
submitted file/URI name in IngestPanel state to pin it in the 'working' status card until
done/error. Pure SPA change + catalog strings (6 locales) + tests.",
  "effort": "S",
  "tracked_in": "khelsutra#99"
}

Repo root candidates: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer,
C:/Users/avidu/Projects/khelsutra-guru/khelsutra, C:/Users/avidu/Projects/khelsutra-guru/sports-
data-collector, C:/Users/avidu/Projects/khelsutra-guru/rally-annotator,
C:/Users/avidu/Projects/khelsutra-guru/sports-obsreport (pick from the finding's repo field;
non-git dirs: 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup',
C:/Users/avidu/Projects/khelsutra-guru/khelsutra-screencast).
VERIFY against the CURRENT working tree (just pulled to origin master/main): (1) does the cited
code actually exist and behave as claimed at file:line? (2) was it already fixed by a recent
commit (git log --oneline -15 -- <file>)? (3) is the severity honest - construct the concrete
failure scenario or downgrade; (4) is the proposed fix sane and correctly scoped; (5) is it
owner-gated per: alpha/serving go-live, Studio P10/P11 corpus-promotion/training,
product/strategy/licensing, real cloud spend, counsel ToS/DPA?
HARD RULES: READ-ONLY (no file modifications, no installs, no test-suite runs). gh read-only
commands allowed. Your reasoning MUST cite file:line you actually read. CONFIRMED requires you
personally saw the defect/debt in current code; if you cannot reproduce the claim from the code,
say REFUTED; if a commit fixed it, ALREADY_FIXED.
```

2. user (2026-07-02T03:02:30.278Z)

```
f9eea14 web: resilient video-summary readers + show fps in the library (#140)
9223ba9 docs: refresh khelsutra docs for today's storage-track work (P2-P5) (#139)
```

```
fb2f561 web: persistent "FFmpeg not detected" banner (hard-blocker warning) (#138)
8667dbf P5: SPA storage-medium picker (store a video into your medium) (#137)
a04a21b docs(tracker): inline the D14 decisions into the P2-P5 rows (#136)
```

master...origin/master

3. user (2026-07-02T03:02:31.443Z)

```
No files found
```

4. user (2026-07-02T03:02:37.846Z)

```
No files found
```

5. user (2026-07-02T03:02:42.964Z)

```
khelsutra\web\src\components\IngestPanel.tsx
```

6. user (2026-07-02T03:02:43.864Z)

```
khelsutra\web\src\hooks\useUpload.ts
```

7. user (2026-07-02T03:02:52.574Z)

```
File does not exist. Note: your current working directory is C:\Users\avidu\Projects\khelsutra-
guru.
```

8. user (2026-07-02T03:03:06.535Z)

```
1       // Drives the in-UI file UPLOAD half of ingest (tracker T4): chunk a local file up to
the engine's
2       // /api/upload/* endpoints, report byte progress, then hand the returned server PATH to
the existing
3       // processing back-half (useIngestJob.submit). Alpha users have a FILE, not a gs:// URI
? this is the
4       // keystone that lets them point the tool at it from the browser.
5       //
6       // Kept i18n-FREE (returns stable IngestErrorCodes via classifyError, not prose) so it
unit-tests
7       // without a translation context ? IngestPanel maps codes to localized strings via
errorText. Abort
8       // is an AbortController: cancel/unmount/a superseding pick stops the in-flight upload
and the
9       // orchestrator best-effort DELETEs the server-side partial.
10      import { useCallback, useEffect, useRef, useState } from "react";
11      import { uploadFile } from "../api/client";
12      import { classifyError, type IngestError } from "../lib/errors";
13      import { log, newCorrelationId } from "../lib/log";
14
15      export type UploadPhase =
16        | { status: "idle" }
17        | { status: "uploading"; pct: number; filename: string }
18        | { status: "error"; error: IngestError };
19
20      export function useUpload(onComplete: (path: string) => void) {
21        const [phase, setPhase] = useState<UploadPhase>({ status: "idle" });
22        const ctrlRef = useRef<AbortController | null>(null);
23        // Keep the latest onComplete without making start depend on it (a parent re-render
won't restart).
24        const onCompleteRef = useRef(onComplete);
25        onCompleteRef.current = onComplete;
26
27        const stop = useCallback(() => {
28          ctrlRef.current?.abort();
```

```
        ctrlRef.current = null;
      }, []);

      // Abort any in-flight upload on unmount.
      useEffect(() => stop, [stop]);

      const start = useCallback(
        (file: File) => {
          // A 0-byte "video" is never valid ? reject before touching the network (reuse
invalidUri copy).
          if (!file || file.size === 0) {
            setPhase({ status: "error", error: { code: "invalidUri" } });
            return;
          }

          stop(); // supersede any prior upload
          const ctrl = new AbortController();
          ctrlRef.current = ctrl;
          const cid = newCorrelationId();
          setPhase({ status: "uploading", pct: 0, filename: file.name });
          log("info", "upload.submit_start", { cid, filename: file.name, size: file.size });

          void (async () => {
            try {
              const res = await uploadFile(file, {
                cid,
                signal: ctrl.signal,
                onProgress: ({ sentBytes, totalBytes }) => {
                  if (ctrl.signal.aborted) return;
                  const pct = totalBytes > 0 ? Math.min(100, Math.round((sentBytes /
totalBytes) * 100)) : 0;
                  setPhase({ status: "uploading", pct, filename: file.name });
                },
              });
              if (ctrl.signal.aborted) return;
              setPhase({ status: "idle" }); // hand off ? the engine processing phase takes
over
              onCompleteRef.current(res.path);
            } catch (err) {
              // User-cancelled (abort) is silent, not an error.
              if (ctrl.signal.aborted || (err instanceof DOMException && err.name ===
"AbortError")) return;
              setPhase({ status: "error", error: classifyError(err) });
              log("error", "upload.failed", { cid, filename: file.name, error: String(err)
});
            }
          })();
        },
        [stop],
      );

      const cancel = useCallback(() => {
        stop();
        setPhase({ status: "idle" });
        log("info", "upload.cancel", {});
      }, [stop]);

      const reset = useCallback(() => {
        stop();
        setPhase({ status: "idle" });
      }, [stop]);

      const busy = phase.status === "uploading";
      return { phase, start, cancel, reset, busy };
    }
```

9. user (2026-07-02T03:03:13.603Z)

```
1       // The front-half ingest screen (appliance S1/S2; tracker B-P2 + T4). Two ways to point
the tool at a
2       // video: UPLOAD a local file (primary ? alpha users have a file, not a URI; chunked to
the engine's
3       // /api/upload/* endpoints) or paste a `gs://` bucket URI / host path (secondary). Both
converge on
4       // the SAME back-half: POST /api/process ? poll GET /api/jobs/{id} ? hand the video_id
to App. Honest
5       // by construction: the engine reports coarse phase WORDS (not a %), and a success-
shaped failure
6       // (done but no rallies) is shown, never silently treated as a load.
7       import { useEffect, useState } from "react";
8       import type { ChangeEvent, FormEvent } from "react";
9       import { useTranslation } from "react-i18next";
10      import { createAsset, getCapabilities } from "../api/client";
11      import type { StorageBackendInfo } from "../api/types";
12      import { useIngestJob } from "../hooks/useIngestJob";
13      import { useUpload } from "../hooks/useUpload";
14      import { classifyError, type IngestError } from "../lib/errors";
15      import { errorText } from "../lib/errorText";
16      import { log, newCorrelationId } from "../lib/log";
17      import { StatusChip, Spinner } from "./StatusChip";
18
19      type Compute = "local" | "cloud";
20      // The three storage-medium choices the picker offers. "bucket" covers s3 OR gcs (the
user pastes a
21      // full scheme'd URI); "drive" is a mounted Google Drive folder
(STUDIO_MEDIA_LIFECYCLE.md P5, D14).
22      type Medium = "local" | "bucket" | "drive";
23
24      interface Props {
25        /** Called with the engine's video_id once a submitted job finishes successfully. */
26        onLoaded: (videoId: string) => void;
27      }
28
29      export function IngestPanel({ onLoaded }: Props) {
30        const { t, i18n } = useTranslation();
31        const [uri, setUri] = useState("");
32        const { phase, submit, cancel: cancelIngest, reset: resetIngest, busy: ingestBusy } =
useIngestJob(onLoaded);
33        // The UI locale recorded with the submission (PMF analytics + the localized reel
watermark).
34        const locale = i18n.resolvedLanguage ?? i18n.language;
35
36        // Compute-picker (PACKAGING compute-picker, item 3): if this server can dispatch to a
cloud GPU
37        // (capabilities.deployment.cloud_available), let the user choose where DETECT runs.
Default LOCAL
38        // (free, no surprise cost); CLOUD requires an explicit cost acknowledgement before it
can run.
39        const [cloudAvailable, setCloudAvailable] = useState(false);
40        const [compute, setCompute] = useState<Compute>("local");
41        const [cloudAck, setCloudAck] = useState(false);
42        // Storage-medium picker (P5): which media THIS box can store into
(capabilities.storage.backends,
43        // usable-only). Absent/local-only ? no picker (today's flow unchanged).
44        const [backends, setBackends] = useState<StorageBackendInfo[]>([]);
45        const [medium, setMedium] = useState<Medium>("local");
46        const [mediumUri, setMediumUri] = useState("");
47        const [storing, setStoring] = useState(false);
48        const [storeError, setStoreError] = useState<IngestError | null>(null);
```

```
       useEffect(() => {
         let alive = true;
         getCapabilities()
           .then((c) => {
             if (!alive) return;
             setCloudAvailable(c.deployment?.cloud_available === true);
             setBackends(c.storage?.backends ?? []);
           })
           .catch((err) => log("warn", "capabilities.unavailable", { error: String(err) }));
         return () => {
           alive = false;
         };
       }, []);
       // Only send a choice when the picker is actually shown; otherwise omit it (? server
default).
       const sendCompute = cloudAvailable ? compute : undefined;
       // CLOUD selected but the cost note not yet accepted ? block submit (don't silently
run local, and
       // don't dispatch a paid job without consent).
       const cloudBlocked = cloudAvailable && compute === "cloud" && !cloudAck;

       // Which medium segments to offer: local always; "bucket" if s3 OR gcs is usable;
"drive" if a Drive
       // mount is usable. The picker only appears when there's a REAL choice (a non-local
usable backend).
       const canUse = (id: string) => backends.some((b) => b.id === id && b.usable);
       const canBucket = canUse("s3") || canUse("gcs");
       const canDrive = canUse("gdrive");
       const mediumOptions: Medium[] = ["local", ...(canBucket ? ["bucket" as const] : []),
...(canDrive ? ["drive" as const] : [])];
       const showMediumPicker = mediumOptions.length > 1;
       const needsMediumUri = medium !== "local";
       // A non-local medium needs a destination URI before anything can be stored/submitted.
       const mediumBlocked = needsMediumUri && mediumUri.trim() === "";

       // Store the obtained source (an uploaded local path, or a pasted at-rest URI) INTO
the chosen medium,
       // then process the durable copy. A failed store is loud (surfaced like every other
ingest error).
       const storeThenProcess = async (src: { uploadedPath?: string; sourceUri?: string }) =>
{
         const cid = newCorrelationId();
         setStoreError(null);
         setStoring(true);
         try {
           const asset = await createAsset({ ...src, mediumUri: mediumUri.trim() }, cid);
           log("info", "asset.stored", { cid, video_id: asset.video_id, medium: asset.medium,
copied: asset.copied });
           submit(asset.stored_uri, locale, sendCompute); // index the stored copy
         } catch (err) {
           setStoreError(classifyError(err));
           log("error", "asset.store_failed", { cid, error: String(err) });
         } finally {
           setStoring(false);
         }
       };

       // On upload completion: local medium ? process the upload directly (today's flow); a
non-local
       // medium ? store a durable copy into it first, then process. The arrow is recreated
each render
       // (useUpload reads onCompleteRef.current), so `medium`/`mediumUri`/`sendCompute` are
always fresh.
       const upload = useUpload((path) => {
         if (medium === "local") submit(path, locale, sendCompute);
```

```
102         else void storeThenProcess({ uploadedPath: path });
103       });
104       const busy = ingestBusy || upload.busy || storing;
105       // Disable the submit surfaces while busy, awaiting the cloud cost ack, or missing a
medium URI.
106       const inputsDisabled = busy || cloudBlocked || mediumBlocked;
107
108       const onSubmitUri = (e: FormEvent) => {
109         e.preventDefault();
110         if (inputsDisabled) return;
111         // The URI form is the SOURCE only in local mode (a bucket URL / host path to index
directly). For
112         // a non-local medium the source is the upload; the text box below is the
DESTINATION, not this.
113         submit(uri, locale, sendCompute);
114       };
115       const onFile = (e: ChangeEvent<HTMLInputElement>) => {
116         const file = e.target.files?.[0];
117         // Symmetric with onSubmitUri: never start while blocked (the disabled input already
prevents this).
118         if (file && !inputsDisabled) upload.start(file);
119         e.target.value = ""; // let re-picking the same file fire change again
120       };
121       const cancelAll = () => {
122         upload.cancel();
123         cancelIngest();
124       };
125       const resetAll = () => {
126         upload.reset();
127         resetIngest();
128         setStoreError(null);
129       };
130
131       // Shared mapping with the build flow (lib/errorText): an `engine` failure carries the
engine's own
132       // message (a 400 scheme/path/extension detail, a worker error); every other code maps
to a friendly
133       // localized line. Surface whichever half failed (upload or engine) ? they're mutually
exclusive.
134       const errorState =
135         upload.phase.status === "error"
136           ? upload.phase.error
137           : storeError ?? (phase.status === "error" ? phase.error : null);
138       const errorMsg = errorState ? errorText(t, errorState) : null;
139
140       return (
141         <section className="mt-6 rounded-2xl border border-black/10 bg-white p-5" data-
testid="ingest-panel">
142           <h2 className="text-lg font-bold">{t("ingest.title")}</h2>
143           <p className="mt-1 max-w-2xl text-sm text-ink/60">{t("ingest.intro")}</p>
144
145           {/* COMPUTE PICKER (item 3): shown only when this server can dispatch to a cloud
GPU. Default
146               LOCAL; CLOUD reveals a cost note + a required acknowledgement before any
submit can run. */}
147           {cloudAvailable && (
148             <fieldset className="mt-4 rounded-xl border border-black/10 p-3" data-
testid="compute-picker">
149               <legend className="px-1 text-xs font-semibold uppercase tracking-wide text-
ink/40">
150                 {t("ingest.compute.legend")}
151               </legend>
152               <div className="mt-1 flex flex-wrap gap-2">
153                 {(["local", "cloud"] as const).map((opt) => (
154                   <button
```

```
                    key={opt}
                    type="button"
                    aria-pressed={compute === opt}
                    disabled={busy}
                    onClick={() => setCompute(opt)}
                    className={
                      "min-h-11 rounded-xl border px-4 py-2 text-sm font-semibold
disabled:opacity-40 " +
                      (compute === opt
                        ? "border-green bg-green/10 text-ink"
                        : "border-black/15 text-ink/70 hover:border-green")
                    }
                  >
                    {t(`ingest.compute.${opt}`)}
                  </button>
                ))}
              </div>
              {compute === "cloud" && (
                <div className="mt-3" data-testid="compute-cost">
                  <p className="text-xs text-ink/60">{t("ingest.compute.cloudCost")}</p>
                  <label className="mt-2 flex items-center gap-2 text-sm text-ink/80">
                    <input
                      type="checkbox"
                      checked={cloudAck}
                      disabled={busy}
                      onChange={(e) => setCloudAck(e.target.checked)}
                      className="h-4 w-4"
                    />
                    {t("ingest.compute.ack")}
                  </label>
                  {cloudBlocked && (
                    <p className="mt-1 text-xs text-amber-700" data-testid="compute-ack-
required">
                      {t("ingest.compute.ackRequired")}
                    </p>
                  )}
                </div>
              )}
            </fieldset>
          )}

          {/* MEDIUM PICKER (P5, D14): where to STORE this video. Rendered only from the
media this box can
              actually use (capabilities.storage.backends, usable-only), and only when
there's a real
              choice beyond local. A non-local medium reveals a destination-URI field; the
video is stored
              there (a durable, MD5-keyed copy ? free-space checked) and then indexed. Self-
host posture. */}
          {showMediumPicker && (
            <fieldset className="mt-4 rounded-xl border border-black/10 p-3" data-
testid="medium-picker">
              <legend className="px-1 text-xs font-semibold uppercase tracking-wide text-
ink/40">
                {t("ingest.medium.legend")}
              </legend>
              <div className="mt-1 flex flex-wrap gap-2">
                {mediumOptions.map((opt) => (
                  <button
                    key={opt}
                    type="button"
                    aria-pressed={medium === opt}
                    disabled={busy}
                    onClick={() => {
                      setMedium(opt);
```

```
212                       setStoreError(null);
213                     }}
214                     data-testid={`medium-${opt}`}
215                     className={
216                       "min-h-11 rounded-xl border px-4 py-2 text-sm font-semibold
disabled:opacity-40 " +
217                       (medium === opt
218                         ? "border-green bg-green/10 text-ink"
219                         : "border-black/15 text-ink/70 hover:border-green")
220                     }
221                   >
222                     {t(`storage.${opt}`)}
223                   </button>
224                 ))}
225               </div>
226               {needsMediumUri && (
227                 <div className="mt-3" data-testid="medium-dest">
228                   <input
229                     type="text"
230                     value={mediumUri}
231                     onChange={(e) => setMediumUri(e.target.value)}
232                     placeholder={medium === "drive" ? t("ingest.medium.drivePlaceholder") :
t("ingest.medium.bucketPlaceholder")}
233                     aria-label={t("ingest.medium.destAria")}
234                     disabled={busy}
235                     className="min-h-11 w-full max-w-md rounded-xl border border-black/15
bg-white px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
236                   />
237                   <p className="mt-2 text-xs text-ink/50">{t("ingest.medium.destHint")}</p>
238                   {mediumBlocked && (
239                     <p className="mt-1 text-xs text-amber-700" data-testid="medium-
required">
240                       {t("ingest.medium.destRequired")}
241                     </p>
242                   )}
243                 </div>
244               )}
245             </fieldset>
246           )}
247
248           {/* PRIMARY: upload a local file (T4). A native file input, visually a button
(?44px, A4). */}
249           <div className="mt-4">
250             <label
251               className={
252                 // focus-within makes the visible button show a focus ring when the (sr-
only) input is
253                 // focused ? a keyboard user needs a visible focus indicator (WCAG 2.4.7).
254                 "inline-flex min-h-11 items-center rounded-xl bg-ink px-4 py-2 text-sm font-
semibold text-white hover:bg-ink2 focus-within:outline-none focus-within:ring-2 focus-
within:ring-green focus-within:ring-offset-2 " +
255                 (inputsDisabled ? "pointer-events-none opacity-40" : "cursor-pointer")
256               }
257             >
258               {t("ingest.chooseFile")}
259               <input
260                 type="file"
261                 accept=".mp4,.mov,video/mp4,video/quicktime"
262                 aria-label={t("ingest.uploadAria")}
263                 disabled={inputsDisabled}
264                 onChange={onFile}
265                 className="sr-only"
266               />
267             </label>
268             <p className="mt-2 text-xs text-ink/40">{t("ingest.fileHint")}</p>
```

```
269           </div>
270
271           {/* SECONDARY: in LOCAL mode, point at a cloud bucket / host path to index
directly. Hidden for a
272               non-local medium, where the destination field above owns the URI box and the
upload is the
273               source (avoids two conflicting URI inputs). */}
274           {medium === "local" && (
275           <div className="mt-4 border-t border-black/5 pt-4">
276             <p className="text-xs font-semibold uppercase tracking-wide text-
ink/40">{t("ingest.orLink")}</p>
277             <form onSubmit={onSubmitUri} className="mt-2 flex flex-wrap items-center gap-2">
278               <input
279                 type="text"
280                 value={uri}
281                 onChange={(e) => setUri(e.target.value)}
282                 placeholder={t("ingest.uriPlaceholder")}
283                 aria-label={t("ingest.uriAriaLabel")}
284                 disabled={inputsDisabled}
285                 className="min-h-11 min-w-[18rem] flex-1 rounded-xl border border-black/15
bg-white px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
286               />
287               <button
288                 type="submit"
289                 disabled={inputsDisabled}
290                 className="min-h-11 rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-
white disabled:opacity-40"
291               >
292                 {phase.status === "submitting" ? t("ingest.submitting") :
t("ingest.submit")}
293               </button>
294             </form>
295             <p className="mt-2 text-xs text-ink/40">{t("ingest.hint")}</p>
296           </div>
297           )}
298
299           {/* Live status (aria-live so screen readers hear progress) as icon+text chips ?
never icon-only
300               (A8). Upload shows a determinate % bar; the engine phase is indeterminate
(phase words). */}
301           <div aria-live="polite" className="mt-3 text-sm" data-testid="ingest-status">
302             {upload.phase.status === "uploading" && (
303               <div data-testid="upload-progress">
304                 {/* Static chip label (not the per-tick %) so the aria-live region announces
once; the live
305                     % rides the progressbar's aria-valuenow + a sighted-only label (A8/T4).
*/}
306                 <StatusChip tone="info" icon={<Spinner />}
label={t("ingest.uploadingStatus")} />
307                 <div className="mt-2 flex max-w-sm items-center gap-2">
308                   <div
309                     className="h-1.5 flex-1 overflow-hidden rounded-full bg-black/10"
310                     role="progressbar"
311                     aria-label={t("ingest.uploadProgress")}
312                     aria-valuenow={upload.phase.pct}
313                     aria-valuemin={0}
314                     aria-valuemax={100}
315                   >
316                     <div className="h-full bg-green transition-all" style={{ width:
`${upload.phase.pct}%` }} />
317                   </div>
318                   <span aria-hidden="true" className="text-xs tabular-nums text-ink/50">
319                     {upload.phase.pct}%
320                   </span>
321                 </div>
```

```
322                 <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
323                 <button
324                   type="button"
325                   onClick={cancelAll}
326                   className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
327                 >
328                   {t("ingest.cancel")}
329                 </button>
330               </div>
331             )}
332             {storing && (
333               <div data-testid="ingest-storing">
334                 <StatusChip tone="info" icon={<Spinner />}
label={t("ingest.medium.storing")} />
335               </div>
336             )}
337             {ingestBusy && (
338               <div>
339                 <StatusChip
340                   tone="info"
341                   icon={<Spinner />}
342                   label={
343                     t("ingest.working") +
344                     (phase.status === "polling" && phase.progress
345                       ? ` · ${t("ingest.progress", { progress: phase.progress })}`
346                       : "")
347                   }
348                 />
349                 {/* Reassurance for slow uplinks: the job runs on the server, so it's safe
to wait. */}
350                 <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
351                 <button
352                   type="button"
353                   onClick={cancelAll}
354                   className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
355                 >
356                   {t("ingest.cancel")}
357                 </button>
358               </div>
359             )}
360             {errorMsg && (
361               <div className="rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-
red-800" data-testid="ingest-error">
362                 <StatusChip tone="error" icon="?" label={t("ingest.statusFailed")} />
363                 <p className="mt-2">{errorMsg}</p>
364                 <button
365                   type="button"
366                   onClick={resetAll}
367                   className="mt-2 inline-flex min-h-11 items-center rounded-lg border
border-red-300 px-3 text-sm hover:bg-red-100"
368                 >
369                   {t("ingest.retry")}
370                 </button>
371               </div>
372             )}
373           </div>
374         </section>
375       );
376     }
377
```

10. user (2026-07-02T03:03:24.061Z)

<tool_use_error>Path does not exist: C:/Users/avidu/Projects/khelsutra-guru/khelsutra/web/src/locales/en/common.json. Note: your current working directory is C:\Users\avidu\Projects\khelsutra-guru.</tool_use_error>

11. user (2026-07-02T03:03:24.959Z)

```
khelsutra\web\src\App.tsx:178:    const filename = reelFilename(reelName);
khelsutra\web\src\App.tsx:192:      filename,
khelsutra\web\src\App.tsx:199:      const res = await compileReel(videoId, ids, filename, cid,
i18n.resolvedLanguage ?? i18n.language);
khelsutra\web\src\App.tsx:200:      const url = highlightUrl(videoId, filename);
khelsutra\web\src\App.tsx:201:      setBuild({ status: "done", url, filename });
khelsutra\web\src\App.tsx:202:      log("info", "build.done", { cid, video_id: videoId,
filename, filepath: res.filepath, url });
khelsutra\web\src\App.tsx:209:      log("error", "build.failed", { cid, video_id: videoId,
filename, status, code: e.code, detail: e.detail, error: String(err) });
khelsutra\web\src\api\locale.test.ts:41:    expect(postBodyOf(f)).toEqual({ video_id: "vid",
segment_ids: [1, 2], output_filename: "r.mp4", locale: "kn" });
khelsutra\web\src\api\locale.test.ts:48:    expect(postBodyOf(f)).toEqual({ video_id: "vid",
segment_ids: [1], output_filename: "r.mp4" });
khelsutra\web\src\api\client.test.ts:152:      output_filename: "reel.mp4",
khelsutra\web\src\api\client.ts:187:    body: JSON.stringify({ video_id: videoId, segment_ids:
segmentIds, output_filename: outputFilename, locale }),
khelsutra\web\src\api\client.ts:223:/** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
khelsutra\web\src\api\client.ts:224:export function highlightUrl(videoId: string, filename:
string): string {
khelsutra\web\src\api\client.ts:225:  return
`${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;
khelsutra\web\src\api\client.ts:248:export async function uploadInit(filename: string,
totalSizeBytes: number, cid?: string): Promise<UploadInitResponse> {
khelsutra\web\src\api\client.ts:251:    body: JSON.stringify({ filename, total_size_bytes:
totalSizeBytes }),
khelsutra\web\src\api\client.ts:323:  log("info", "upload.start", { cid, filename: file.name,
size: file.size, parts: totalParts, part_mb: init.max_part_mb });
khelsutra\web\src\i18n\catalog-invariants.test.ts:34:// renders inline <code> (the
rally_annotator.lua filename, the ending_reason enum, <video>.rallies.csv)
khelsutra\web\src\components\a8.status.test.tsx:33:    render(<CompileResult build={{ status:
"done", url: "/x", filename: "r.mp4" }} onDismiss={() => {}} />);
khelsutra\web\src\components\a8.status.test.tsx:40:    render(<CompileResult build={{ status:
"done", url: "/x", filename: "r.mp4" }} onDismiss={() => {}} />);
khelsutra\web\src\components\a8.status.test.tsx:50:    render(<CompileResult build={{ status:
"done", url: "/x", filename: "r.mp4" }} onDismiss={() => {}} />);
khelsutra\web\src\hooks\useUpload.ts:17:  | { status: "uploading"; pct: number; filename: string
}
khelsutra\web\src\hooks\useUpload.ts:47:      setPhase({ status: "uploading", pct: 0, filename:
file.name });
khelsutra\web\src\hooks\useUpload.ts:48:      log("info", "upload.submit_start", { cid,
filename: file.name, size: file.size });
khelsutra\web\src\hooks\useUpload.ts:58:              setPhase({ status: "uploading", pct,
filename: file.name });
khelsutra\web\src\hooks\useUpload.ts:68:          log("error", "upload.failed", { cid, filename:
file.name, error: String(err) });
khelsutra\web\src\i18n\hygiene.test.ts:51:// {filename}/{n}/? and break interpolation. (ICU
plural headers like `{count, plural, ?}` carry no
khelsutra\web\src\components\CompileResult.tsx:12:  | { status: "done"; url: string; filename:
string }
khelsutra\web\src\components\CompileResult.tsx:71:              download={build.filename}
khelsutra\web\src\components\CompileResult.tsx:74:              {t("result.download", {
filename: build.filename })}
khelsutra\web\src\lib\format.ts:15:/** Turn a user reel name into a safe bare `.mp4` filename
for POST /api/compile. The engine's
khelsutra\web\src\lib\format.ts:16: * `safe_output_filename` (backend/utils/paths.py) rejects
path separators / traversal / absolute
```

12. user (2026-07-02T03:03:32.289Z)

```
<tool_use_error>Path does not exist: C:/Users/avidu/Projects/khelsutra-
guru/khelsutra/web/src/locales. Note: your current working directory is
C:\Users\avidu\Projects\khelsutra-guru.</tool_use_error>
```

13. user (2026-07-02T03:03:32.909Z)

```
270         <main className="min-h-screen bg-paper text-ink pb-24">
271           <header className="bg-ink text-white">
272             <div className="mx-auto flex max-w-5xl items-center justify-between gap-3 px-6
py-4">
273               <span className="text-xl font-extrabold tracking-tight">
274                 Khel<span className="text-green">?</span>Sutra <span className="font-
semibold text-lime">Studio</span>
275               </span>
276               <LanguageSwitcher />
277             </div>
278           </header>
279
280           <section className="mx-auto max-w-5xl px-6 py-10">
281             {/* Hard-blocker warning: FFmpeg absent ? indexing + reel compile can't run.
Persistent (unlike
282                 the once-per-browser wizard), shown only when the engine positively reports
it missing. */}
283             <FfmpegBanner />
284             <span className="inline-flex rounded-full bg-green/15 px-3 py-1 text-xs font-
semibold text-green">
285               {t("app.badge")}
286             </span>
287             <h1 className="mt-4 text-3xl font-extrabold tracking-
tight">{t("app.title")}</h1>
288             <p className="mt-2 max-w-2xl text-ink/70">
289               <Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} />
290             </p>
291
292             {/* Persistent "current video" label ? always visible once a video is loaded, so
the operator
293                 never loses track of which match they're editing (the library/ingest panel
that showed the
294                 name has unmounted by now). Name from the library, else resolved from GET
/api/videos. */}
295             {videoId && (
296               <p
297                 className="mt-3 inline-flex items-center gap-2 rounded-full bg-ink/5 px-3
py-1 text-sm text-ink/70"
298                 data-testid="current-video"
299               >
300                 <span aria-hidden="true">?</span>
301                 <Trans
302                   i18nKey="app.currentVideo"
303                   values={{ name: videoLabel ?? t("app.currentVideoUnknown") }}
304                   components={{ b: <strong className="text-ink" /> }}
305                 />
306               </p>
307             )}
308
309             {/* P8 (flagged): switch the loaded video between picking rallies and annotating
them. */}
310             {videoId && annotateEnabled && (
311               <div className="mt-4 inline-flex rounded-full border border-black/10 p-1 text-
sm" data-testid="view-toggle">
312                 <button
313                   type="button"
314                   onClick={() => setView("pick")}
```

```
315                   aria-pressed={view === "pick"}
316                   className={"min-h-11 rounded-full px-4 " + (view === "pick" ? "bg-ink
text-white" : "text-ink/70")}
317                 >
318                   {t("annotate.tabPick")}
319                 </button>
```

14. user (2026-07-02T03:03:40.523Z)

```
No files found
```

15. user (2026-07-02T03:03:43.151Z)

```
8667dbf P5: SPA storage-medium picker (store a video into your medium) (#137)
f67dfe7 feat(ingest): compute-picker ? choose local vs cloud GPU per run (item 3b) (#104)
c1848df feat(web): LW2 ? send the UI locale to /api/compile (watermark) + /api/process
(analytics) (#73)
ca619fb feat(web): T4 ? in-UI file upload (the runnable-tool keystone) (#66)
92a6c45 feat(web): A8 ? background-processing reassurance + icon+text status chips (#63)
501c014 feat(web): A5 ? friendly engine-error mapping across build + load (shared errors.*)
(#62)
95195a5 feat(web): A4 ? mobile touch targets (?44px) + visible tooltip text (#61)
4525c5d feat(web): B-P2 ? in-UI bucket ingest (point at a gs:// video ? process ? load) (#56)
---
{"labels":[],"state":"OPEN","title":"SPA: keep the uploaded video's filename visible during
processing (it disappears mid-job)","updatedAt":"2026-06-
22T14:01:52Z","url":"https://github.com/avidullu/khelsutra/issues/99"}
```

16. user (2026-07-02T03:03:51.237Z)

```
locales/da/common.json
locales/en/common.json
locales/es/common.json
locales/hi/common.json
locales/id/common.json
locales/kn/common.json
site/public/locales/da/common.json
site/public/locales/en/common.json
site/public/locales/es/common.json
site/public/locales/hi/common.json
site/public/locales/id/common.json
site/public/locales/kn/common.json
site/scripts/copy-locales.mjs
site/src/locales-drift.test.js
```

17. user (2026-07-02T03:03:59.297Z)

```
locales/da/common.json:95:    "uploadingStatus": "Uploader?",
locales/en/common.json:95:    "uploadingStatus": "Uploading?",
locales/es/common.json:95:    "uploadingStatus": "Subiendo?",
locales/hi/common.json:95:    "uploadingStatus": "????? ?? ??? ???",
locales/id/common.json:95:    "uploadingStatus": "Mengunggah?",
locales/kn/common.json:95:    "uploadingStatus": "???????? ??????????",
site/public/locales/da/common.json:95:    "uploadingStatus": "Uploader?",
site/public/locales/en/common.json:95:    "uploadingStatus": "Uploading?",
site/public/locales/es/common.json:95:    "uploadingStatus": "Subiendo?",
site/public/locales/hi/common.json:95:    "uploadingStatus": "????? ?? ??? ???",
```

18. user (2026-07-02T03:04:40.282Z)

```
Structured output provided successfully
```